

Introdução à Programação em C: Aula 01

COMANDO.C

1 A Linguagem da Máquina: Binário

Na base de tudo, o computador entende apenas sequências de 1s e 0s (binário):

Linguagem de máquina

```
11000111010001011111010000000101
000000000000000000000000011000111
010001011111110000000011100000000
```

O bloco de 1s e 0s acima é um exemplo de **código de máquina**, escrito na linguagem de máquina: binário. Cada linha é uma instrução que contém o tipo da operação, o local e os dados.

Para que um programa escrito em uma linguagem de qualquer nível possa ser executado por um computador, ele precisa ser traduzido para a linguagem que o hardware entende. As máquinas digitais trabalham internamente com sinais elétricos representando apenas dois estados possíveis: ligado (1) e desligado (0). Esses dois estados formam o “alfabeto” do computador.

Uma boa analogia é comparar esse alfabeto com o nosso: embora o português tenha apenas 26 letras, isso não limita o número de ideias que podemos expressar. Da mesma forma, o computador utiliza apenas duas “letras” — 0 e 1 — mas isso não limita sua capacidade de processamento ou o tipo de problemas que ele pode resolver. É a combinação desses símbolos que permite construir estruturas complexas.

Cada um desses símbolos é chamado de bit (dígito binário). Quando agrupamos bits, formamos padrões que representam números, caracteres, cores, sons e, principalmente, instruções.

Essas instruções são conjuntos específicos de bits que o processador é capaz de reconhecer e executar. Elas representam operações como somar valores, mover dados na memória, comparar números ou desviar o fluxo do programa. Essa coleção de instruções forma a linguagem de máquina, que é a forma mais simples e direta de comunicação com o hardware.

O papel da linguagem C — e de qualquer outra linguagem — é justamente permitir

ao programador escreva essas instruções de forma compreensível, enquanto o compilador realiza a tradução para os padrões de bits que o processador precisa para operar.

2 O que é Linguagem de Programação?

Como vimos na seção anterior, escrever em binário (linguagem de máquina) não é prático para nós, humanos. Seria como tentar escrever um livro complexo usando apenas os dígitos 0 e 1.

Para resolver isso, criamos as **Linguagens de Programação**.

Uma linguagem de programação é uma **ponte**: ela é um idioma formal, com regras rígidas, que fica no meio-termo. Ela é compreensível para o programador, mas também foi projetada para ser facilmente traduzida (pelo compilador) para o código de máquina que o computador entende.

Abstração: o Grande Poder

Pense na última vez em que usou um microondas para esquentar seu almoço. Certamente não lhe foi um desafio tão grande, afinal você só precisou seguir uma lista simples de passos:

- Pegar seu prato;
- Abrir a porta do microondas;
- Colocar o prato lá dentro;
- Fechar a porta e escolher por quanto tempo o prato deve ficar;
- E, por fim, aguardar o tempo acabar.

Dessa forma, seu delicioso prato está pronto para ser consumido! Mas veja que curioso, o usuário desse aparelho não precisou, em nenhum momento, aprofundar-se no seu funcionamento, como seus circuitos garantem a segurança de operação e sequer os fundamentos físicos da matéria que explicam o porquê de sua comida realmente esquentar.

Isso nada mais é que a **abstração**, uma interface simples que esconde algo complexo por trás. E em programação isso também ocorre, pois é o que torna viável, para nós humanos, desenvolver um software completo sem olhar para 1s e 0s.



No quadro acima, vemos que o código binário passa por camadas de abstração, distanciando-se cada vez mais de seu estado original — o código de máquina — até chegar a uma

linguagem mais inteligível para o cérebro humano. São essas camadas de abstração que garantem que a computação como conhecemos hoje exista. Todo e qualquer conceito básico que permeie essa área só é possível por conta desse conceito.

3 Compilador: do Código Fonte ao Executável

Acreditamos que, até esse momento, você tenha entendido que as camadas de abstração permitem que uma pessoa programe um sistema inteiro sem precisar escrever em linguagem de máquina.

Entretanto, o binário é o único tipo de instrução que um computador consegue compreender. Mas como seu código C é convertido em linguagem de máquina?

```
1000110010100000
```

Tomemos como exemplo o código binário acima, que diz exatamente ao computador que deve somar dois números. Para chegarmos a esse resultado criamos uma linguagem simbólica que pode ser traduzida exatamente para os mesmo dígitos citados — processo esse garantido pelos *montadores* (*assembler*), programas que traduzem um código em linguagem abstrata como esse

```
add A, B
```

para seu equivalente em instrução binária, de modo que o computador será capaz de executá-lo. A linguagem abstrata, que chamaremos daqui por diante de *linguagem de montagem*, que usamos nessa instrução se chama Assembly, ela é a linguagem mais próxima da máquina que você irá encontrar, exceto o próprio binário, pois suas instruções são traduzidas uma por uma em instruções equivalentes a linguagem de máquina.

Graças ao Assembly, somos capazes de ler e compreender nosso código. No entanto, sua sintaxe é diretamente atrelada à forma como as instruções são entendidas pelo computador, não sendo tão próxima da nossa linguagem natural. Para contornar esse problema, foram desenvolvidas as linguagens de programação, e a que vamos usar em nossas aulas é o **C**!

O programa que traduz seu código da linguagem de alto nível para linguagem de montagem e, por fim, para a linguagem de máquina, chama-se *compilador*. Ele lê cada linha do seu código, confere se a **sintaxe** está correta, traduz para assembly e, posteriormente, gera um arquivo de extensão *.obj*. Esse arquivo, após receber instruções adicionais — pelos *linkeditores* —, é transformado em um executável (extensão *.exe*). *Para mais informações sobre o funcionamento dos compiladores, confira nosso apêndice de referências.*

Programa
em linguagem
de alto
nível (em C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

↓
Compilador

Programa
em
linguagem
Assembly
(para o
MIPS)

```
swap:
  multi $2, $5,4
  add   $2, $4,$2
  lw    $15, 0($2)
  lw    $16, 4($2)
  sw    $16, 0($2)
  sw    $15, 4($2)
  jr    $31
```

↓
Assembler

Programa
em linguagem
de máquina
binária (para
o MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
10101110000100100000000000000000
101011011110001000000000000000100
0000001111100000000000000000001000
```

Figura 1: Esquema adaptado de Patterson & Hennessy (Organização e Projeto de Computadores).

Apêndice A: Algoritmo vs Programa

Nem todo algoritmo é um programa, apesar de usarmos como sinônimos. **Algoritmo** é um passo a passo, pode ser descrito em português ou em uma receita, enquanto o **programa** é a implementação desse algoritmo em qualquer linguagem de programação.

Algoritmo – Receita de bolo:

1. Bata a clara em neve e reserve.
2. Misture as gemas, a margarina e o açúcar até obter uma massa homogênea.
3. Acrescente o leite e a farinha de trigo aos poucos, sem parar de bater.
4. Por último, adicione as claras em neve e o fermento.
5. Despeje a massa em uma forma grande de furo central untada e enfarinhada.
6. Asse em forno médio 180 °C, preaquecido, por aproximadamente 40 minutos ou ao furar o bolo com um garfo, este saia limpo.

Programa 1: Exemplo de programa em C (Soma dois números)

```
1  #include <stdio.h>
2  int main() {
3      int x, y;
4      printf("\nInforme um valor para x: ");
5      scanf("%d", &x);
6      printf("\nInforme um valor para y: ");
7      scanf("%d", &y);
8      printf("\nSoma dos valores fornecidos: %d\n", x+y);
9      return 0;
10 }
```

Apêndice: Referências

Why Some Projects Use Multiple Programming Languages — Core Dumped

